

Corner Store Assistant

Practice exercise: an agent with tools, memory, streaming and tracing

1. Objective

Build an interactive terminal assistant for a small store. The agent must rely on tools for every fact (stock and prices) instead of guessing, remember what was said earlier in the conversation, print its answers while they are being generated, and leave a clear trail in the OpenAI traces dashboard. You will practice agent creation, system prompts, the `@function_tool` decorator, the Runner, conversation memory, streaming and tracing.

2. Store data

Keep the store data in a plain Python dictionary. No external API is needed. Prices are in AED.

Product	Price (AED)	Stock
milk	7.00	12
bread	4.50	0
eggs	12.00	8
apples	3.00	30
coffee	25.00	5

3. Functional requirements: tools

The agent has three tools. Each one must have a clear docstring, because the model reads it to decide when to call the tool.

Tool	Input	Behavior
check_stock	product name	Returns how many units are in stock. Reports clearly when the product does not exist in the store.
get_price	product name	Returns the unit price. Reports clearly when the product does not exist.
calculate_total	list of items, each with a product name and a quantity	Returns the total cost of the basket by multiplying each unit price by its quantity.

4. Business rules for the system prompt

Write a system prompt that makes the agent follow these rules. How you word them is up to you.

#	Rule
1	The agent is a friendly assistant for the store, with a short and polite tone.
2	It must never state a price or stock level from its own knowledge. Every number comes from a tool or from an earlier tool result in the conversation.
3	If a product does not exist, it says so politely and mentions what the store does sell.
4	If the customer asks for more units than are in stock, it tells them the available quantity and does not confirm the full order.
5	Questions unrelated to the store are declined politely, without calling any tool.
6	Follow-up messages such as "how much for 5 of them?" or "add 2 coffees to that" are understood from the earlier conversation. If the reference is truly unclear, the agent asks which product the customer means.

5. Memory requirements

The assistant runs as a chat loop in the terminal and keeps going until the user types exit. Every user message and every agent reply of the session must be stored, and on every turn the agent must receive the earlier messages as context so follow-up questions work. Choose one of the two options below. SQLite is the more ambitious choice because the conversation can survive a restart.

Option	Expected behavior
Local list of dicts	Each message is stored as a dictionary with at least a role and the content. The memory lives only while the script is running.
SQLite	Each message is saved in a table with a session id, role, content and timestamp. When the script starts, the user can type an existing session id and continue that conversation, even after a restart.

6. Streaming requirements

The agent's reply must appear in the terminal progressively, as it is generated, and not all at once at the end. When the agent starts using a tool, the terminal shows a short line with the tool name and its arguments, and another indication when the tool has finished. After the stream ends, the complete reply is saved to memory exactly like in the non-streaming version.

7. Tracing requirements

Every chat session must be visible in the OpenAI traces dashboard under a custom workflow name of your choice, for example Corner Store Assistant. Each turn produces its own trace, but all turns of the same session must be linked by a shared identifier so you can filter them together. Use the same session id for memory and for tracing, so a conversation stored in your memory can always be found in the dashboard.

8. Non-functional requirements

The script runs from the terminal. The API key is read from an environment variable and never written in the code. A failing tool or a network error must not crash the chat loop: the user sees a short error message and can keep chatting.

9. Example conversations

These examples show the behavior you should aim for. Tool calls and results are what you should see in the trace. The exact wording of the agent will differ from run to run, but the tool usage and the numbers should match.

Example A: chaining two tools

YOU	Do you have milk, and how much would 3 cost?
TOOL CALL	<code>check_stock(product="milk")</code>
TOOL RESULT	12 units in stock
TOOL CALL	<code>get_price(product="milk")</code>
TOOL RESULT	7.00 AED
TOOL CALL	<code>calculate_total(items=[{"product": "milk", "quantity": 3}])</code>
TOOL RESULT	21.00 AED
AGENT	Yes, we have milk in stock! It costs 7.00 AED each, so 3 bottles come to 21.00 AED. Anything else I can help with?

Example B: basket with several products

YOU	I want 2 cartons of eggs and 1 coffee. What is the total?
TOOL CALL	<code>check_stock(product="eggs")</code>
TOOL RESULT	8 units in stock
TOOL CALL	<code>check_stock(product="coffee")</code>
TOOL RESULT	5 units in stock
TOOL CALL	<code>calculate_total(items=[{"product": "eggs", "quantity": 2}, {"product": "coffee", "quantity": 1}])</code>
TOOL RESULT	49.00 AED
AGENT	Both are available. 2 eggs at 12.00 AED plus 1 coffee at 25.00 AED gives a total of 49.00 AED.
NOTE	The model may call the tools one after another or several in the same step. Check the trace to see which it chose.

Example C: not enough stock

YOU	Can I buy 20 bottles of milk?
TOOL CALL	<code>check_stock(product="milk")</code>
TOOL RESULT	12 units in stock
AGENT	Sorry, we only have 12 bottles of milk right now, so I cannot confirm 20. Would you like 12 instead?

Example D: out of stock

YOU	How much is a loaf of bread?
TOOL CALL	<code>get_price(product="bread")</code>
TOOL RESULT	4.50 AED
TOOL CALL	<code>check_stock(product="bread")</code>
TOOL RESULT	0 units in stock
AGENT	Bread costs 4.50 AED, but unfortunately it is out of stock at the moment. Can I help you with something else?

Example E: product that does not exist

YOU	Do you sell bananas?
TOOL CALL	<code>check_stock(product="bananas")</code>
TOOL RESULT	Product not found
AGENT	I am sorry, we do not carry bananas. We do have milk, bread, eggs, apples and coffee. Would any of these work?

Example F: off-topic question

YOU	What is the weather like today?
AGENT	I am only able to help with questions about our store, such as prices and availability. Is there anything you would like to know about our products?
NOTE	No tool is called. If the trace shows one, your system prompt needs to be stricter.

Example G: memory across turns (one session)

YOU	Do you have apples?
TOOL CALL	<code>check_stock(product="apples")</code>
TOOL RESULT	30 units in stock
AGENT	Yes, we have apples in stock.
YOU	How much for 5 of them?
TOOL CALL	<code>get_price(product="apples")</code>
TOOL RESULT	3.00 AED
TOOL CALL	<code>calculate_total(items=[{"product": "apples", "quantity": 5}])</code>
TOOL RESULT	15.00 AED
AGENT	5 apples come to 15.00 AED.
YOU	Add 2 coffees to that and give me the new total.
TOOL CALL	<code>check_stock(product="coffee")</code>
TOOL RESULT	5 units in stock
TOOL CALL	<code>calculate_total(items=[{"product": "apples", "quantity": 5}, {"product": "coffee", "quantity": 2}])</code>
TOOL RESULT	65.00 AED
AGENT	Done! 5 apples and 2 coffees come to 65.00 AED in total.
NOTE	In the last two turns the customer never repeats the product or the quantity. The agent can only answer because the earlier messages were sent with each turn. Without memory it should ask what "them" refers to.

Example H: resuming a session (SQLite only)

NOTE	The script was closed and started again, and the customer typed the same session id from Example G.
YOU	What was my last total?
AGENT	Your last total was 65.00 AED, for 5 apples and 2 coffees. Would you like to change anything?
NOTE	No tool call is needed, because the number comes from an earlier tool result stored in the history.

Example I: how the terminal should look while streaming

```
you> Do you have milk, and how much would 3 cost?  
[tool] check_stock(product="milk") ... done  
[tool] get_price(product="milk") ... done  
[tool] calculate_total(items=[{"product": "milk", "quantity": 3}]) ... done  
agent> Yes, we have milk in stock! It costs 7.00 AED each, so 3 bottles come to 21.00 AED.
```

The tool lines appear the moment each tool runs, and the agent text then grows word by word instead of appearing in one block.

10. Checking the traces on the OpenAI platform

After running the examples, open the traces dashboard and go through the points below. This is the most valuable part of the exercise.

What to check	What it tells you
Workflow name and shared session identifier	All turns of one chat appear together when you filter by the session id. Confirm that a new session gets a different identifier.
Order of spans inside a turn	The sequence of model calls and tool calls. Compare Example A (three tools) with Example F (no tools).
Arguments generated for the tools	In Example G, open the turn "How much for 5 of them?" and confirm the model filled in apples on its own. This proves the memory reached the model.
Input of the model call in a later turn	Open the model call of the third turn in Example G and confirm that the earlier messages are part of its input.
Duration of each span	Where the time goes: model thinking, tool execution, or the extra round trip after the last tool result.
A streamed turn versus the final text	Confirm that the streamed answer you saw in the terminal matches the output recorded in the trace.

11. Experiments

Try at least two of these once the main exercise works.

Experiment	What to observe
Make the system prompt vague, then very strict	Does the agent still call tools for every number? Does it start guessing prices?
Remove the docstring from one tool	Does the model still choose the right tool? How do the arguments change?
Ask with typos ("mlik") or in another language	How does the agent handle a product name that does not match exactly?
Send only the last 4 messages as memory, then ask a follow-up that depends on the first turn	When does the agent lose the context? How does the trace input change?
Disable memory completely and repeat Example G	Does the agent ask for clarification or does it guess?
Add a fourth tool, <code>apply_discount(code, total)</code> , where the code <code>SAVE10</code> gives 10% off	Does the agent use it on its own after "I have the code <code>SAVE10</code> "? On the 65.00 AED basket the expected result is 58.50 AED.

12. Done when

Examples A to G (and H if you chose SQLite) behave as described, the answers stream in the terminal, every session is visible in the traces dashboard under your workflow name with its turns grouped, and you can explain from the trace why each tool was called and how memory influenced the later turns.